

Computer Vision(Assignment)

Name : Manoj V

Class : 4th year - CSE - 'B'

1. Apply spatial filtering to enhance the surface of an automotive part image and highlight scratches or dents.

```
# auto_enhance_highlight.py
# pip install opencv-python numpy matplotlib
import cv2, numpy as np, matplotlib.pyplot as plt, os

INPUT_IMAGE = "/Users/veerasenthilkumarr/Documents/Manoj/HCL/M
VTec AD/metal_nut/test/bent/008.png" # update path
os.makedirs("out", exist_ok=True)

# ---- Load image ----
img = cv2.imread(INPUT_IMAGE)
if img is None:
    raise FileNotFoundError("Image not found at given path!")

# ---- Step 1: Sharpen (Unsharp Mask) ----
blur = cv2.GaussianBlur(img, (9,9), 10)
sharpen = cv2.addWeighted(img, 1.5, blur, -0.5, 0)

# ---- Step 2: Edge Detection (Laplacian) ----
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
lap = cv2.Laplacian(gray, cv2.CV_16S, ksize=3)
lapa = cv2.convertScaleAbs(lap)

# ---- Step 3: Threshold & Morphology ----
_, edge_mask = cv2.threshold(lapa, 30, 255, cv2.THRESH_BINARY)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3,3))
edge_mask = cv2.morphologyEx(edge_mask, cv2.MORPH_OPEN, kern
el, iterations=1)
edge_mask = cv2.dilate(edge_mask, kernel, iterations=1)
```

```

# ---- Step 4: Find contours of scratches ----
contours, _ = cv2.findContours(edge_mask, cv2.RETR_EXTERNAL, cv2.
CHAIN_APPROX_SIMPLE)

highlighted = sharpen.copy()
cv2.drawContours(highlighted, contours, -1, (0,0,255), 2) # red contour
s

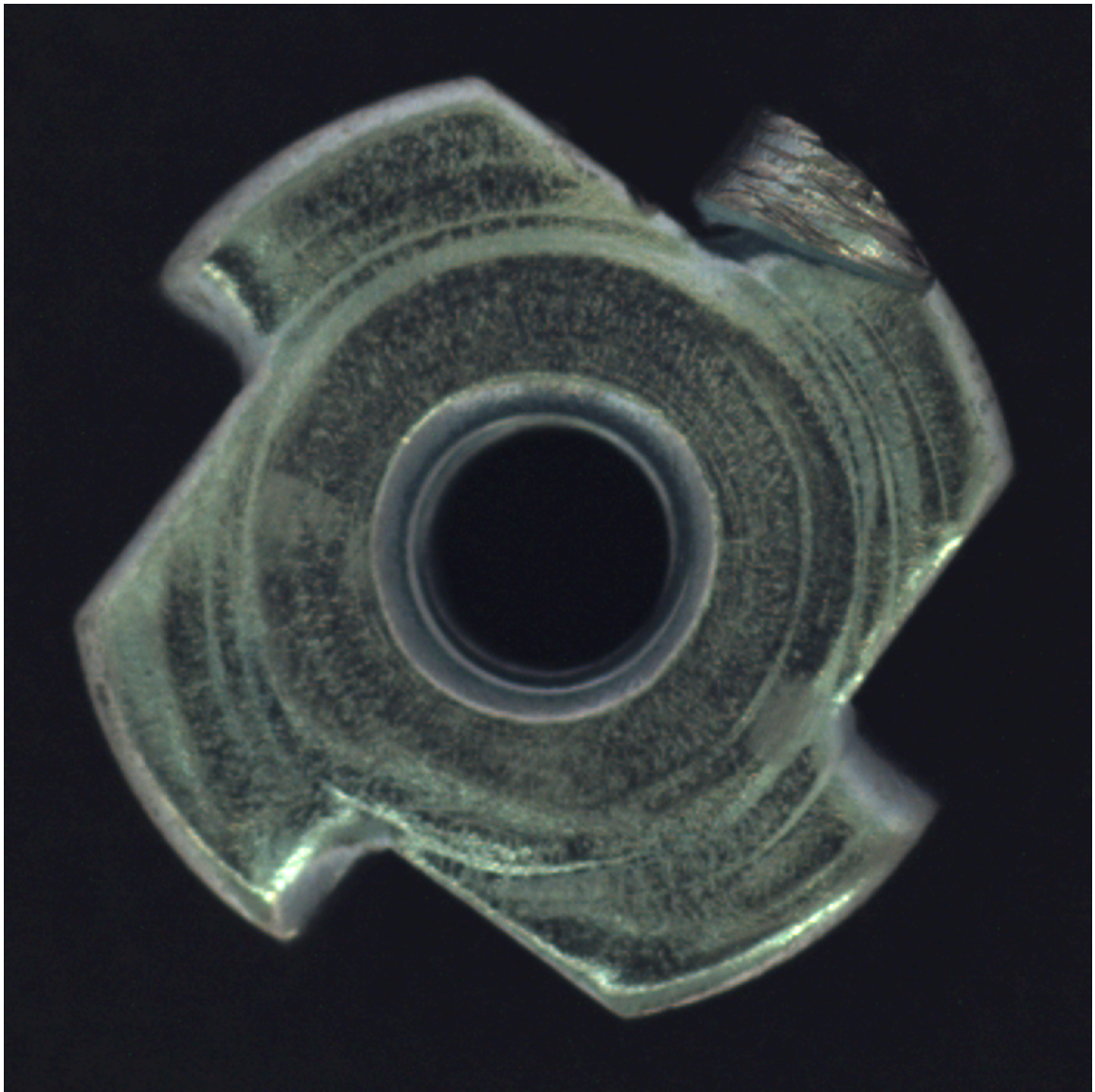
# ---- Save Outputs ----
cv2.imwrite("out/original.png", img)
cv2.imwrite("out/sharpen.png", sharpen)
cv2.imwrite("out/highlighted.png", highlighted)

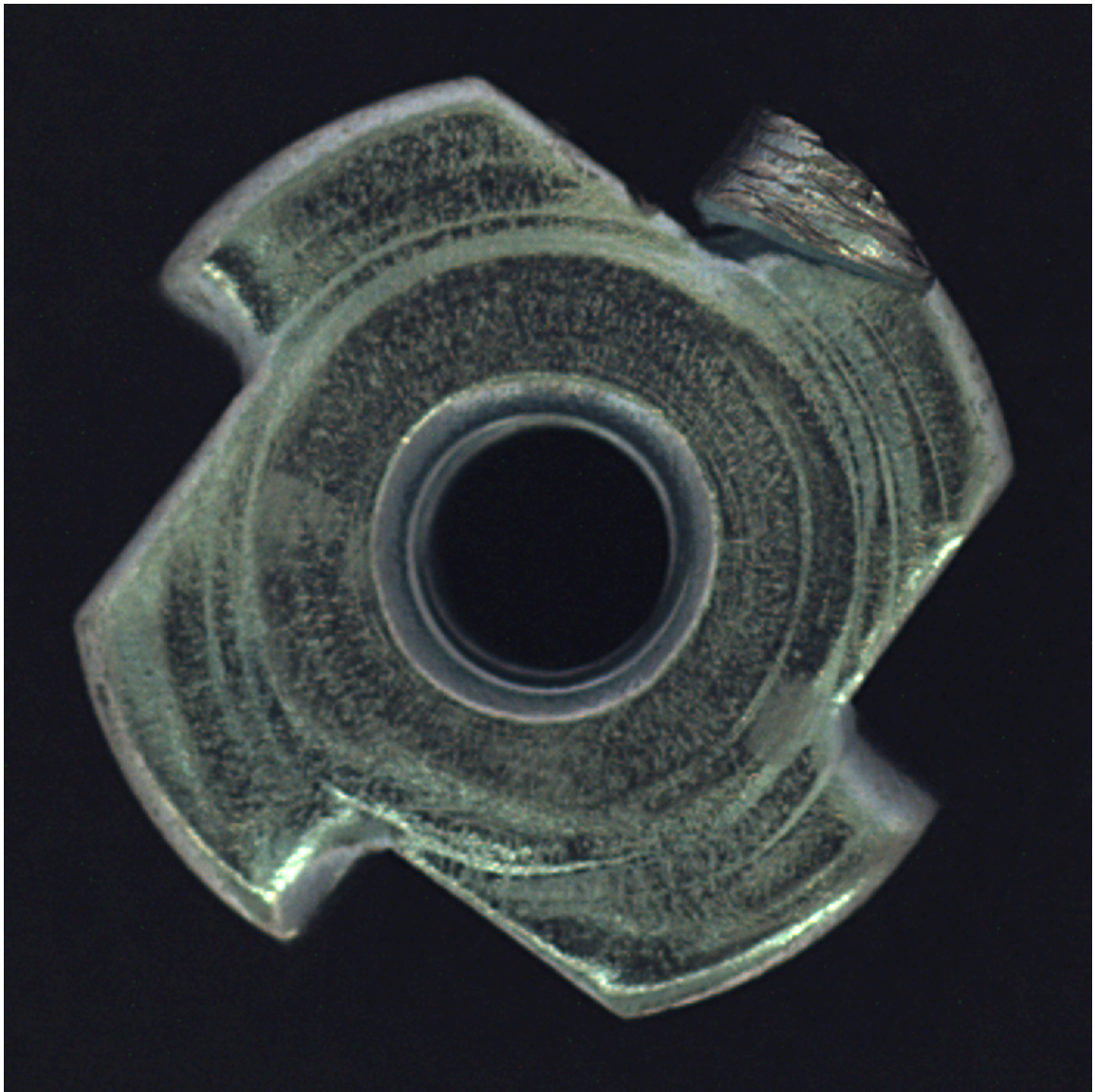
print("Saved → out/original.png, out/sharpen.png, out/highlighted.pn
g")

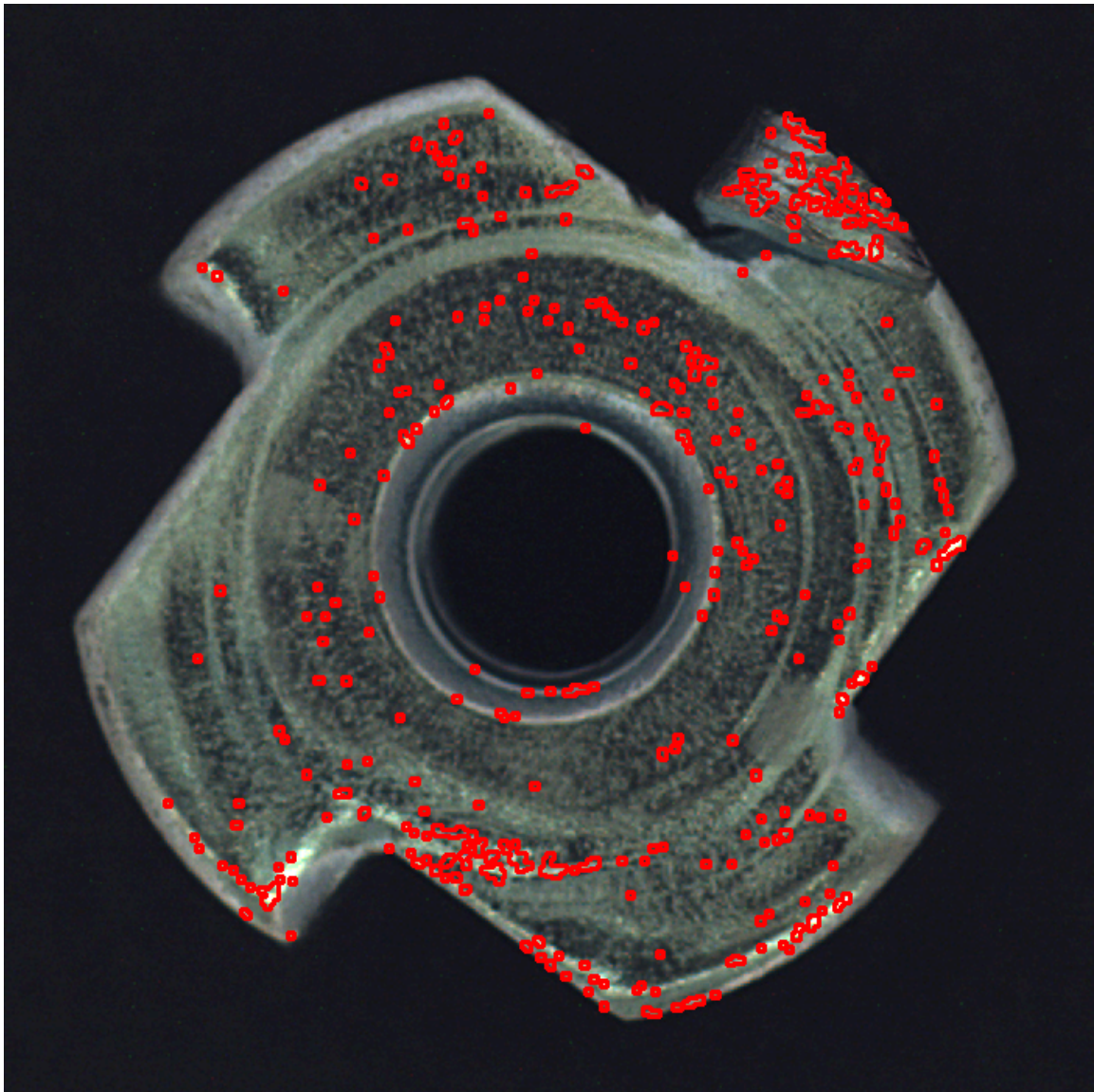
# ---- Show side-by-side ----
titles = ["Original", "Sharpened", "Defects Highlighted"]
images = [cv2.cvtColor(img, cv2.COLOR_BGR2RGB),
          cv2.cvtColor(sharpen, cv2.COLOR_BGR2RGB),
          cv2.cvtColor(highlighted, cv2.COLOR_BGR2RGB)]

plt.figure(figsize=(15,6))
for i in range(3):
    plt.subplot(1,3,i+1)
    plt.imshow(images[i])
    plt.title(titles[i])
    plt.axis("off")
plt.tight_layout()
plt.show()

```







2. Implement Sobel, Canny, and Laplacian edge detectors on bridge/concrete images to detect cracks. Compare their effectiveness

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load automotive part image (replace with your own)
img = cv2.imread("/Users/veerasenthilkumarr/Documents/Manoj/HCL/MVT
ec AD/tile/test/crack/000.png", cv2.IMREAD_GRAYSCALE)

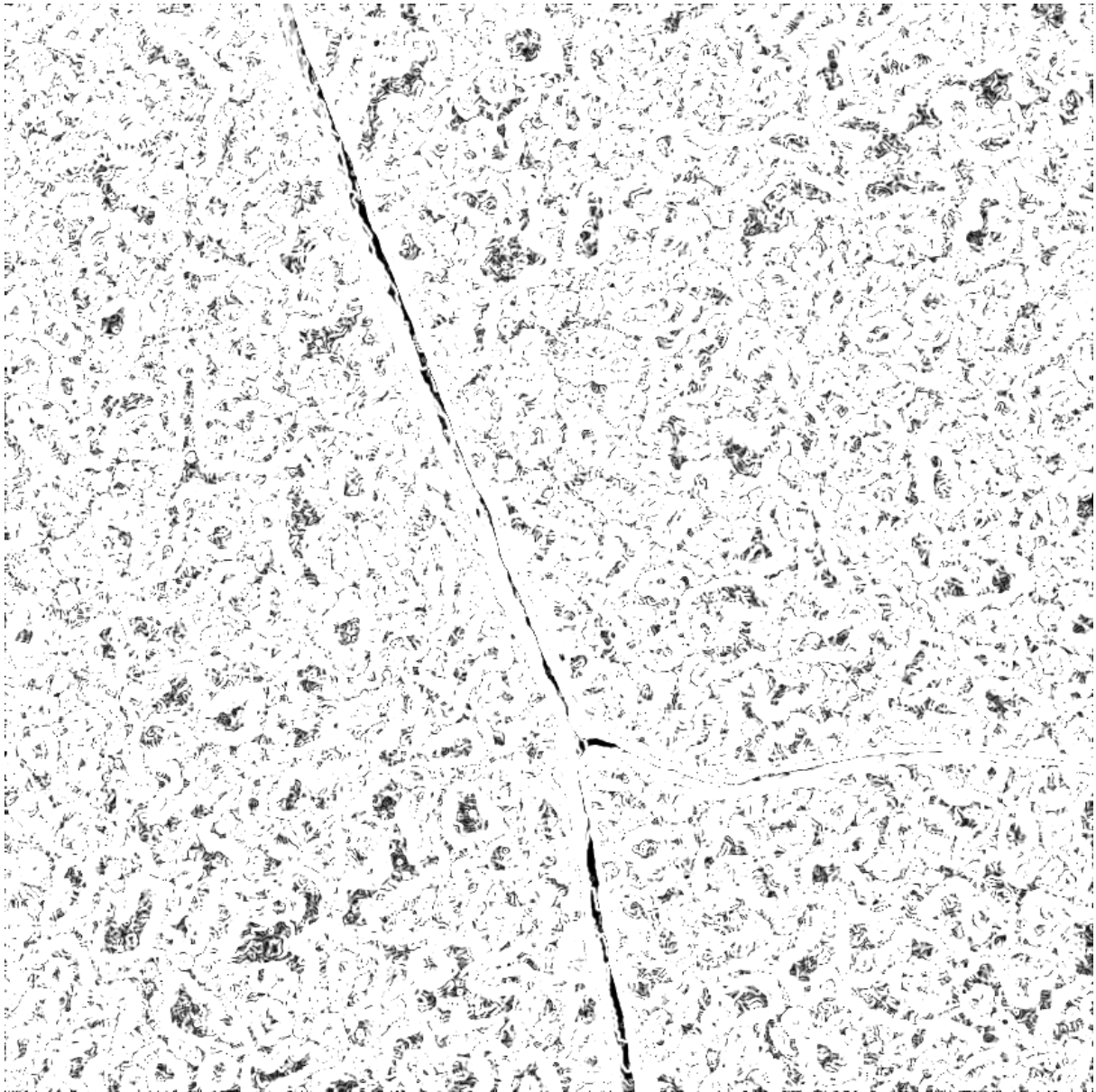
# Apply Sobel filter
sobelx = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=5)
```

```
sobely = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=5)
sobel = cv2.magnitude(sobelx, sobely)

# Apply Laplacian filter
laplacian = cv2.Laplacian(img, cv2.CV_64F)
img_blurred = cv2.GaussianBlur(img, (5, 5), 0)

# Now, run Canny on the BLURRED image
# We can also increase the thresholds a bit
canny_clean = cv2.Canny(img_blurred, 80, 200)
# Show results
cv2.imwrite("auto_sobel.png", sobel)
cv2.imwrite("auto_laplacian.png", laplacian)
cv2.imwrite("auto_canny.png", canny_clean)
```

Output:







3. Use morphological operations (opening/closing) to isolate defective regions in a fabric image

```
import cv2
import numpy as np

img = cv2.imread("/Users/veerasenthilkumarr/Documents/Manoj/HCL/
MVTec AD/leather/test/cut/003.png", 0)
_, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY_INV)

kernel = np.ones((5,5), np.uint8)
opening = cv2.morphologyEx(thresh, cv2.MORPH_OPEN, kernel)
```

```
closing = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)
```

```
cv2.imwrite("fabric_opening.png", opening)
```

```
cv2.imwrite("fabric_closing.png", closing)
```





4. Apply global, adaptive, and Otsu thresholding on metal surface images to separate defective vs. non-defective regions. Provide comparative results

```
import cv2
import numpy as np

# Load original grayscale
img = cv2.imread("/Users/veerasenthilkumarr/Documents/Manoj/HCL/
MVTec AD/metal_nut/test/bent/010.png", 0)

# Global threshold
_, global_thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)

# Adaptive Mean
```

```

adaptive_mean = cv2.adaptiveThreshold(img, 255,
                                     cv2.ADAPTIVE_THRESH_MEAN_C,
                                     cv2.THRESH_BINARY, 11, 2)

# Otsu
_, otsu = cv2.threshold(img, 0, 255,
                       cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# Resize all images to same shape (for stacking)
h, w = img.shape
global_thresh = cv2.resize(global_thresh, (w, h))
adaptive_mean = cv2.resize(adaptive_mean, (w, h))
otsu = cv2.resize(otsu, (w, h))

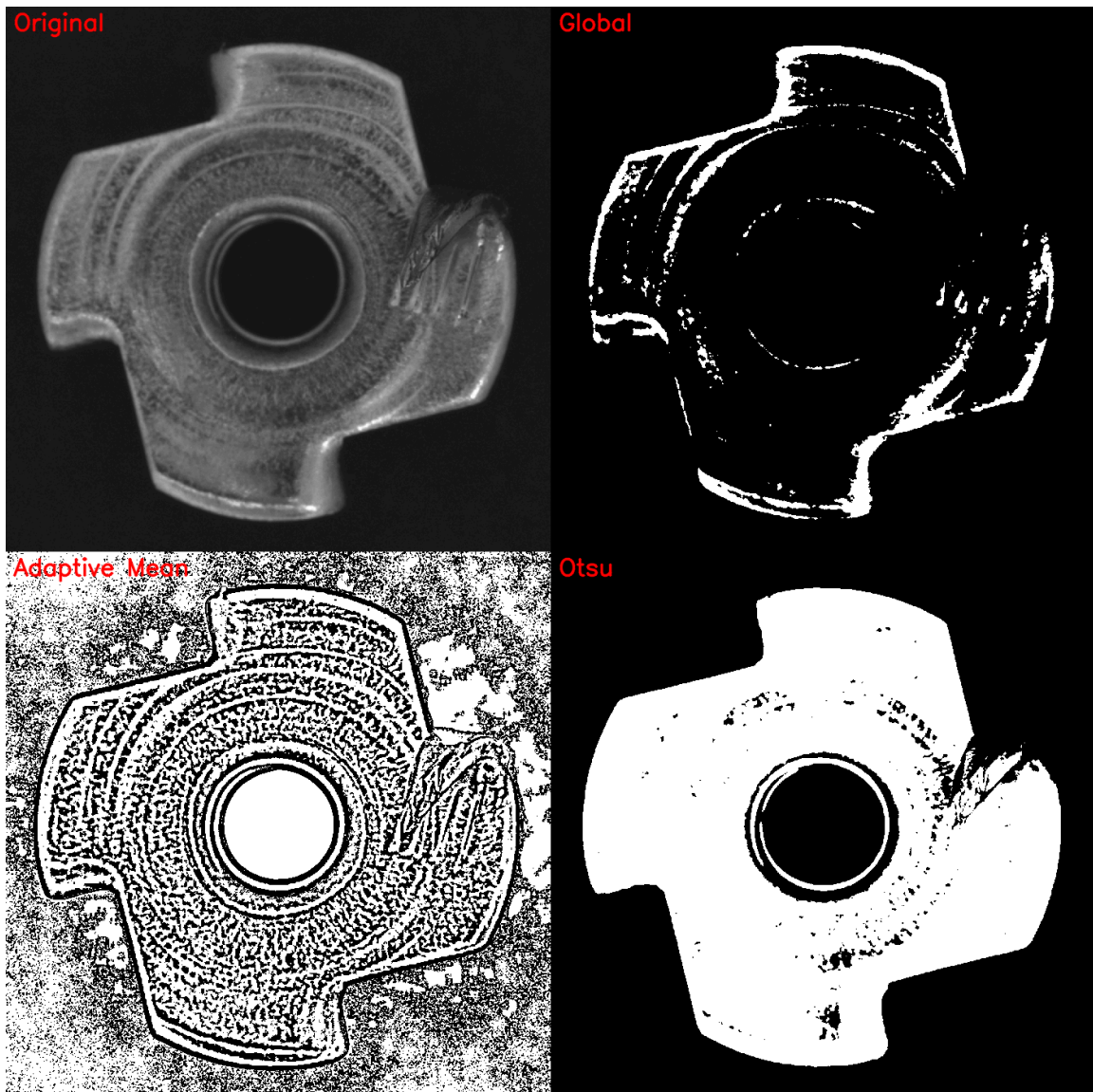
# Convert grayscale to BGR (so we can put text labels)
def add_label(image, label):
    out = cv2.cvtColor(image, cv2.COLOR_GRAY2BGR)
    cv2.putText(out, label, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
                1, (0, 0, 255), 2, cv2.LINE_AA)
    return out

orig = add_label(img, "Original")
gbl = add_label(global_thresh, "Global")
adpt = add_label(adaptive_mean, "Adaptive Mean")
ots = add_label(otsu, "Otsu")

# Stack in 2x2 grid
top_row = cv2.hconcat([orig, gbl])
bottom_row = cv2.hconcat([adpt, ots])
montage = cv2.vconcat([top_row, bottom_row])

# Save final comparative image
cv2.imwrite("metal_threshold_comparison.png", montage)
print("Saved → metal_threshold_comparison.png")

```



5. Develop a color-based thresholding method (HSV space) to classify fruits as ripe or unripe. Test on at least 5 samples.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Define HSV thresholds for ripe and unripe fruits (example for banana
s)
# Adjust these ranges based on the fruit type
ripe_lower = np.array([20, 100, 100]) # Yellow range
ripe_upper = np.array([30, 255, 255])

unripe_lower = np.array([35, 50, 50]) # Green range
```

```

unripe_upper = np.array([85, 255, 255])

def classify_fruit(image_path):
    # Load image
    image = cv2.imread(image_path)
    image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

    # Create masks for ripe and unripe
    ripe_mask = cv2.inRange(image_hsv, ripe_lower, ripe_upper)
    unripe_mask = cv2.inRange(image_hsv, unripe_lower, unripe_upper)

    # Count pixels in each mask
    ripe_count = cv2.countNonZero(ripe_mask)
    unripe_count = cv2.countNonZero(unripe_mask)

    total_count = image.shape[0] * image.shape[1]
    ripe_percent = (ripe_count / total_count) * 100
    unripe_percent = (unripe_count / total_count) * 100

    classification = "Ripe" if ripe_percent > unripe_percent else "Unripe"

    # Plotting
    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1)
    plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    plt.title("Original Image")
    plt.axis('off')

    plt.subplot(1,3,2)
    plt.imshow(ripe_mask, cmap='gray')
    plt.title(f"Ripe Mask ({ripe_percent:.2f}%)")
    plt.axis('off')

    plt.subplot(1,3,3)
    plt.imshow(unripe_mask, cmap='gray')
    plt.title(f"Unripe Mask ({unripe_percent:.2f}%)")
    plt.axis('off')

```

```
plt.suptitle(f"Classification: {classification}", fontsize=16)
plt.show()

return classification

# Test on 5 sample images
sample_images = [
    '/content/images-4.jpeg',
    '/content/images-5.jpeg',
    '/content/images-6.jpeg',
    '/content/green-banana.jpg',
    '/content/shutterstock_518328943-scaled-e1684783243402.jpg'
]

for img_path in sample_images:
    result = classify_fruit(img_path)
    print(f"Image {img_path}: Classified as {result}")
```

Classification: Ripe

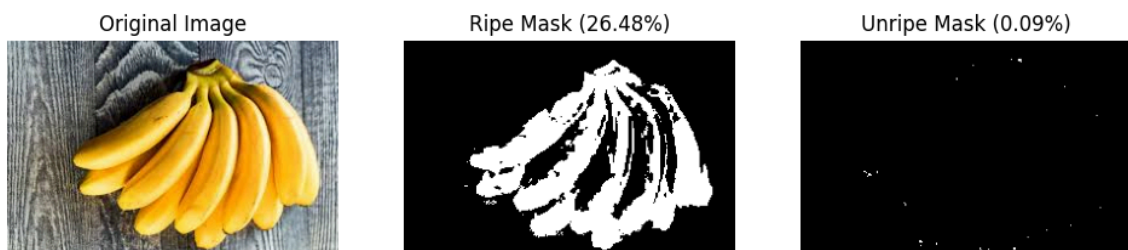


Image /content/images-4.jpeg: Classified as Ripe

Classification: Ripe



Image /content/images-5.jpeg: Classified as Ripe

Classification: Unripe



Image /content/images-6.jpeg: Classified as Unripe

Classification: Unripe



Image /content/green-banana.jpg: Classified as Unripe

Classification: Ripe



Image /content/shutterstock_518328943-scaled-e1684783243402.jpg:
Classified as Ripe

-
6. Use connected components analysis on a tablet image to detect missing, broken, or extra objects. Display bounding boxes for identified defects.

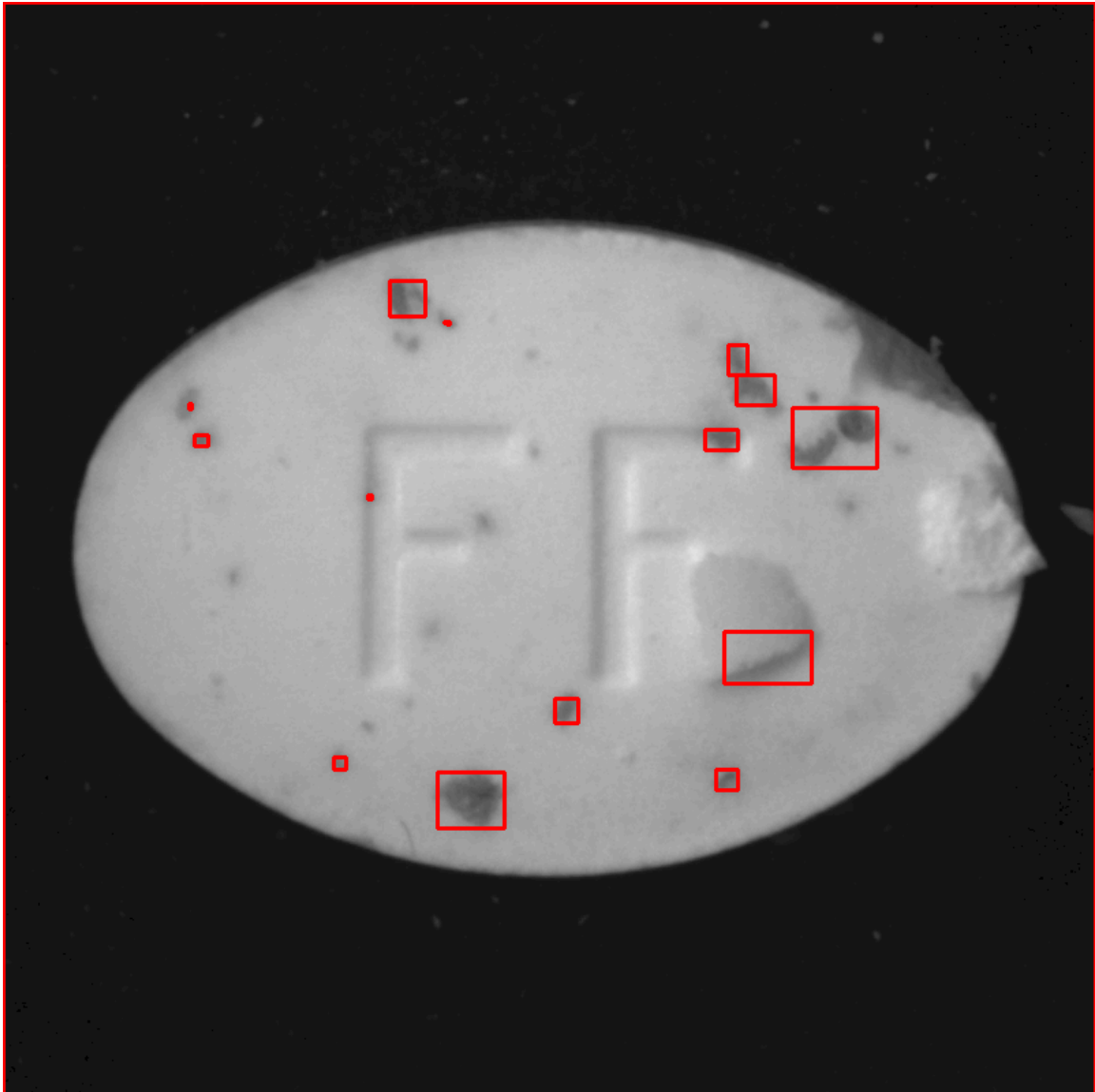
```
import cv2

img = cv2.imread("/Users/veerasenthilkumarr/Documents/Manoj/HCL/
MVTec AD/pill/test/combined/016.png", 0)
_, binary = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY_INV)

# Connected components
num_labels, labels, stats, centroids = cv2.connectedComponentsWithSt
ats(binary, connectivity=8)

output = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
for i in range(1, num_labels):
    x, y, w, h, area = stats[i]
    cv2.rectangle(output, (x, y), (x+w, y+h), (0, 0, 255), 2)

cv2.imwrite("tablets_detected.png", output)
```



7. Implement SIFT or ORB feature matching to detect brand logos/serial numbers in product images. Compare with template matching

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

def load_and_blur(path, blur_size=(5, 5)):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Error loading image: {path}")
    return cv2.GaussianBlur(img, blur_size, 0)
```

```

def draw_keypoints(img, keypoints, title):
    return cv2.drawKeypoints(img, keypoints, None, flags=cv2.DRAW_M
ATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

def show_image(ax, img, title):
    ax.imshow(img, cmap='gray')
    ax.set_title(title)
    ax.axis('off')
template_path = '/Users/veerasenthilkumarr/Downloads/LaptopProduct
2.jpg'
test_path = '/Users/veerasenthilkumarr/Downloads/LaptopTemplate.pn
g'

blur1 = load_and_blur(template_path)
blur2 = load_and_blur(test_path)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(blur1, None)
kp2, des2 = sift.detectAndCompute(blur2, None)

img1_sift = draw_keypoints(blur1, kp1, 'Template - SIFT')
img2_sift = draw_keypoints(blur2, kp2, 'Test - SIFT')

orb = cv2.ORB_create()
kp1_orb, des1_orb = orb.detectAndCompute(blur1, None)
kp2_orb, des2_orb = orb.detectAndCompute(blur2, None)

img1_orb = draw_keypoints(blur1, kp1_orb, 'Template - ORB')
img2_orb = draw_keypoints(blur2, kp2_orb, 'Test - ORB')

print(f"ORB keypoints in template: {len(kp1_orb)}, in test: {len(kp2_or
b)}")

good_matches = []
if des1 is not None and des2 is not None:
    index_params = dict(algorithm=1, trees=5)
    search_params = dict(checks=50)
    flann = cv2.FlannBasedMatcher(index_params, search_params)

```

```

matches = flann.knnMatch(des1, des2, k=2)

for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

sift_matches_img = cv2.drawMatches(blur1, kp1, blur2, kp2, good_matches, None, flags=2)

if des1_orb is not None and des2_orb is not None and len(des1_orb) > 0 and len(des2_orb) > 0:
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
    matches_orb = bf.match(des1_orb, des2_orb)
    matches_orb = sorted(matches_orb, key=lambda x: x.distance)
    orb_matches_img = cv2.drawMatches(blur1, kp1_orb, blur2, kp2_orb, matches_orb[:20], None, flags=2)
else:
    orb_matches_img = np.zeros((max(blur1.shape[0], blur2.shape[0]), blur1.shape[1] + blur2.shape[1], 3), dtype=np.uint8)

result_img = cv2.cvtColor(blur2, cv2.COLOR_GRAY2BGR)
if len(good_matches) > 4: # Need at least 4 points for homography
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    if H is not None:
        h, w = blur1.shape
        corners = np.float32([[0, 0], [0, h], [w, h], [w, 0]]).reshape(-1, 1, 2)
        transformed_corners = cv2.perspectiveTransform(corners, H)
        cv2.polylines(result_img, [np.int32(transformed_corners)], True, (0, 255, 0), 3, cv2.LINE_AA)
        # Draw circle at the center
        cx, cy = np.mean(transformed_corners, axis=0).ravel()
        cv2.circle(result_img, (int(cx), int(cy)), 15, (0, 0, 255), 3)

```

```

fig, axs = plt.subplots(2, 2, figsize=(15, 10))
show_image(axs[0, 0], img1_sift, 'Template - SIFT Keypoints')
show_image(axs[0, 1], img2_sift, 'Test - SIFT Keypoints')
show_image(axs[1, 0], img1_orb, 'Template - ORB Keypoints')
show_image(axs[1, 1], img2_orb, 'Test - ORB Keypoints')
plt.tight_layout()
plt.show()

fig, axs = plt.subplots(1, 3, figsize=(20, 6))
axs[0].imshow(sift_matches_img)
axs[0].set_title('SIFT Matches')
axs[0].axis('off')

axs[1].imshow(orb_matches_img)
axs[1].set_title('ORB Matches')
axs[1].axis('off')

axs[2].imshow(result_img)
axs[2].set_title('Detected Template Location')
axs[2].axis('off')

plt.tight_layout()

save_path = '/Users/veerasenthilkumarr/Documents/Manoj/HCL/Computer Vision/out/SIFT_ORB_Results.png'
os.makedirs(os.path.dirname(save_path), exist_ok=True)

plt.savefig(save_path, bbox_inches='tight')
plt.show()
print(f"Results saved to {save_path}")

```

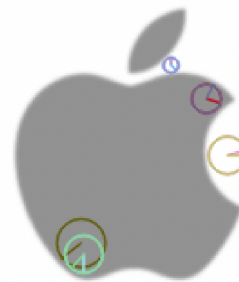
Template - SIFT Keypoints



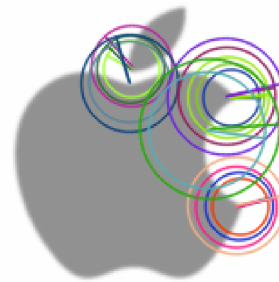
Template - ORB Keypoints



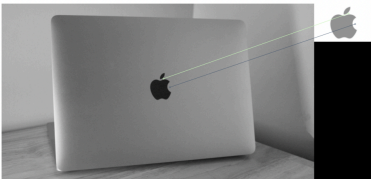
Test - SIFT Keypoints



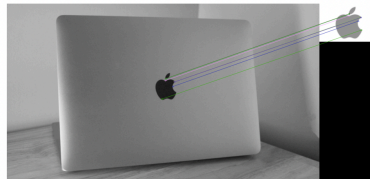
Test - ORB Keypoints



SIFT Matches



ORB Matches



Detected Template Location

